Preparing for the In-Class Midterm

The **in-class midterm** will be held on the following days, based on your section:

- **Section 01:** Monday, November 2, 2026
- **Section 02:** Thursday, November 5, 2026

The purpose of the midterm is twofold:

- First, to assess your comprehension of the course topics from Weeks 1 through 8, and their interrelationships, as they relate to the goals of modern data-driven organizations
- Second, to perform this assessment not in the abstract but via the development of a **concrete plan** that will then serve as a key step in your Final Project!

Your first step, in preparing for the midterm, is to **read through the dataset vignettes** and decide which one is most interesting to you. Once you have decided, **register your decision on Canvas** by choosing an option within the "Midterm Dataset Choice" assignment.

Finally, you can think through how the following "pieces" of what we've learned up to this point in the course would work relative to your chosen dataset!

Question 1: OLTP Infrastructure

This question is **started for you** in the vignette for your chosen dataset, where (in the subsection with header "OLTP Structure") you can get a sense for the **existing OLTP infrastructure** at the organization you've been hired at. In each case, your job is to think through **transforming** the information in this OLTP setup into the kinds of insights/inferences described at the beginning of the vignette.

Question 2: Ingesting, Transforming, and Loading OLTP Snapshots

Sample Question (NYC Shelter Data)

Although events are entered into the OLTP DB "on the fly" in this case, one issue is that receptionists are asked to *double-check* the daily entries towards the end of their shifts, for things like data entry errors.

Given this information, which of the following OLTP-to-OLAP ingestion plans would make best use of the available information without affecting the OLTP system's ability to store frequently-entered records? Think especially in terms of scalability: will the plan still satisfy these criteria if NYC decides to open up many more shelters in the near future? *(Circle **all** answers that would satisfy these goals)*

(a) Run analytic queries (in DuckDB, Polars, and/or Spark) **directly on the OLTP database**, so that the results of these queries always reflect the most up-to-date data available

(b) Take *"snapshots"* of the OLTP records at the very end of each desk worker's shift, and "dump" each of these snapshots into an **S3 bucket**. Then allow data analysts to run analytic queries on the collection of snapshots in this bucket (e.g., using Athena)

(c) Take *"snapshots"* of the OLTP records at the very end of each desk worker's shift, then use an SQL query to store the records contained in each snapshot into a centralized **fact table** with a pre-determined schema. Then allow data analysts to run analytic queries on this centralized fact table

(d) Every time a desk worker enters a record, immediately store the information from the record (e.g., in `.json` format) as a single file within an **S3 bucket**. Then allow data analysts to run analytic queries on the individual files in the S3 bucket using, e.g., Athena

(e) Every time a desk worker enters a record, immediately execute an SQL query to store the information from the record as a new row within a centralized **fact table** with a pre-determined schema. Then allow data analysts to run analytic queries on this centralized fact table.

The two answers you would want to circle here are **(b)** and **(c)**: You can worry about the differential benefits and drawbacks of data **warehouses** vs. data **lakes** in Question 3, but for this question, the key would be to take the snapshots at the end of the shifts so that the **extraction** of the OLTP records doesn't interfere with the actual **operation** of the OLTP system itself!

Option (a) violates this principle, since (as the Boar Book puts it):

> Database administrators closely guard their OLTP databases. They are usually reluctant to let business analysts run ad hoc analytic queries on an OLTP database, since those queries are often expensive, scanning large parts of the dataset, which can harm the performance of concurrently executing transactions.

Options (d) and (e), on the other hand, are worse than options (b) and (c) just because they "clash" with the **columnar** data-storage approach that we typically desire for OLAP infrastructures: to understand this, think of the discussion in Weeks 2-4 about how writing **individual rows** into a **columnar** format like a `.parquet` file or a DuckDB table is extremely computationally expensive: it requires "hopping" around the file many times. Thus, one of the primary benefits of taking **"snapshots"** of the OLTP DB is so that we capture many records at a time, and thus can add many values to the end of a given column before having to "hop" to the next column.

Question 3: OLAP Infrastructure

Sample Question (Enron Email Corpus)

As you can see from the example emails in the vignette, both the content of the emails themselves and their metadata vary wildly in terms of what might be streaming into the OLTP DB (which is just recording the raw text+metadata for every incoming and outgoing email): for example, sometimes the content of the email is included as simple, human-readable plaintext, but sometimes the email body essentially just "points to" a set of attached files. Given this variability, and given how frequently emails are streaming in at this 10,000+ employee organization, which of the following conclusions would you draw for a "best-fit" OLAP infrastructure? *(Choose one)*

(a) A Data Lake infrastructure where all raw data from each email is "dumped" into an S3 bucket (for downstream schema-on-read parsing) makes most sense, since the bandwidth required to write the emails into an S3 bucket would be significantly lower than the bandwidth required to write data from the emails into a warehouse DB table.
(b) A Data Lake infrastructure where all raw data from each email is "dumped" into an S3 bucket (for downstream schema-on-read parsing) makes most sense, since it's difficult to plan in advance what specific "pieces" of the emails might be relevant.
(c) A Data Warehouse infrastructure where email data is written into a table with a pre-defined schema makes most sense, since we can determine in advance what data we want to record and pre-allocate columns within the warehouse DB accordingly
(d) A Data Warehouse infrastructure where email data is written into a table with a pre-defined schema makes most sense, since we can just allocate a "misc_data" column (of raw binary type) to hold any data from the emails that may not fit within the pre-defined schema

Here the answer would be (b), since it's the better choice of infrastructure paired with the better justification for using that infrastructure. (a) is "close" to the right answer, but wouldn't be marked as correct because it's an irrelevant justification (writing into a data warehouse DB doesn't require any more or less bandwidth than writing into a data lake S3 bucket, in general!)

Question 4: Querying OLAP Data

Question 5: Scaling OLAP Queries

Question 6: Serving and Reporting Our Scaled OLAP Query Results